AE 413, Moncayo

Analysis and Simulation of an Aircraft

Embry-Riddle Aeronautical University

Matthew Liepke
4-23-2021

Abstract

Aircraft performance is final test for a designed aircraft; does it meet the required specifications, or is a re-work required? To determine the performance of aircraft there are many methods, both theoretical and empirical to numerically calculate how an aircraft is expected to respond in flight. In this report a mixture of both methods will be used to analyze an aircraft that was designed by an aerospace engineering student in less than 15 hours for a class assessment. This analysis will compare theoretical stability control derivatives calculated by hand and via Digital Datcom, (Digital Datcom Program) and will then implement these stability derivatives in a simulated environment to see how the aircraft performs when tested by a certified pilot.

For the convenience of grading, the locations of each deliverable can be found below. It is important to note that some calculations have multiple locations for each deliverable. A good example are the stability derivatives, where the stability derivatives formulas utilized are shown in Longitudinal Stability Derivatives, implemented in MATLAB in Static Longitudinal Stability Derivatives, and the output values are shown in Appendix II – MATLAB Output.

Table 1: Location of all Project Deliverables

	Weight	Done?	Primary Location	Secondary Location	Tertiary Location
Static Long. Stability	10%	✓	Longitudinal Stability Derivatives	Static Longitudinal Stability Derivatives	Appendix II – MATLAB Output
Static Direc. Stability	10%	✓	Directional Stability Derivatives	Static Directional Stability Derivatives	Appendix II – MATLAB Output
Static Lat. Stability	10%	✓	Lateral Stability Derivatives	Static Lateral Stability Derivatives	Appendix II – MATLAB Output
Static Long. Control	10%	✓	Control Derivatives	Static Control Derivitives	Appendix II – MATLAB Output
FlightGear Interface	5%	✓	FlightGear Interface	'Simulink/LiepkePlane.slx'	'FG.bat'
DATCOM Results	15%	✓	DATCOM Results	Appendix II – DATCOM Input File	Generated and deleted when running 'DatcomImport.m'
DATCOM vs Hand Calculations	5%	✓	Stability Derivative Comparison – DATCOM vs MATLAB		
Lookup Table Integration	15%	✓	Lookup Tables	'LiepkePlane.slx'	'DatcomImport.m'
Short Report	20%	✓	This entire document		

Table of Contents

List of Figures

List of Tables

Introduction

In this report a structured process will be displayed. First, the aircraft to be simulated will be introduced in completion. The main characteristics of it will be displayed and then a table, which will be used in conjunction with theoretical and empirical formulas to calculate major stability derivatives of the aircraft. These values are then going to be input into a simulation environment where they will all be analyzed.

In the testing of the designed aircraft it was found that the aircraft is characteristically stable, albeit twitchy in response. At high speeds the aircraft does develop oscillatory functions for yaw, which although are stable, are not structurally appropriate for piloting. This oscillatory nature in the yaw direction would need to be fixed with an increasingly sized horizontal tail or a more smooth fuselage that does not taper to an extremely thin tail support structure. This could also be remedied with a 'soft line' control system that dampens the input to the rudder and allows for it to 'flap' slightly in the wind. Such a control mechanism would dampen the yaw without major design changes, although it would be an unconventional solution.

Aircraft Geometry and Properties

Because this analysis is supposed to be performed on the aircraft designed during Test 1, first the properties of the aircraft will be introduced. For the characteristics of all surfaces, see Table 2. For the positioning of those surfaces, see

	Units	Value	Notes
x_a	ft	0.5	long dist. From wing AC -> CG
l_f	ft	35.0	length of fuselage
S_f	(--)	120.0	surface area of fuselage in the vertical-drag plane
df_{max}	ft	6.0	maximum depth of fuselage in the span direction

Table 5. For the characteristics of the NACA 1408 airfoil chosen to meet the design requirements of the wing and horizontal tail, see Table 6. For the vertical tail, it was chosen to choose an airfoil of similar properties that was not cambered; the simplest way to do this was to choose the NACA 0008. Although not completely accurate, it was assumed that the NACA 0008 airfoil had identical C_{l_α} to the NACA 1408.

Table 2: Characteristics of the Lifting Surfaces

	Units	Value
AR	(--)	6.9321
AR_v	(--)	2.4500
AR_h	(--)	3.1250
b	ft	30.0000
b_v	ft	7.0000
b_h	ft	5.0000
S	ft^2	130.0000
S_v	ft^2	20.0000
S_h	ft^2	8.0000
Λ_{LE}	rad	0.2618

$\Lambda_{LE,v}$	rad	0.0873
$\Lambda_{LE,h}$	rad	0.2243
λ	(--)	1.0000
λ_{v}	(--)	0.6000
λ_{h}	(--)	0.7000
Γ	rad	-0.0349

Table 3: Control Surface Characteristics / Sizing

	Units	Value
$c_{a_{root}}$	ft	1.5
$c_{a_{tip}}$	ft	1.5
$y_{a_{root}}$	ft	10.0
$y_{a_{tip}}$	ft	15.0
S_r	ft^2	4.0
S_e	ft^2	1.5

Table 4: Fuselage Characteristics

	Units	Value	Notes
x_a	ft	0.5	long dist. From wing AC -> CG
l_f	ft	35.0	length of fuselage
S_f	(--)	120.0	surface area of fuselage in the vertical-drag plane
df_{max}	ft	6.0	maximum depth of fuselage in the span direction

Table 5: Positioning of the Vertical and Horizontal Tail

	Units	Value	Notes
h_h	ft	3.0	vert dist. from horz. tail to wing
l_h	ft	18.0	long dist. from horz. tail AC -> wing AC
η_h	(--)	1.0	
l_v	ft	18.0	long dist. from vert tail to wing
z_w	ft	1.0	horz. distance from wing -> vert tail
z_v	ft	3.1	horz. distance from centerline -> vtail. AC
η_v	(--)	1.0	

Table 6: Characteristics of the NACA 1408 (Airfoil Tools)

	Units	Value
C_{l_0}	(--)	.1147
C_{d_0}	(--)	0.00395
$C_{l_{max}}$	(--)	1.3005
C_{l_α}	deg^{-1}	.1182

With all the values, it is important to visualize the aircraft to get a better picture. Using software provided by Dr. Moncayo, different views of the designed aircraft can be seen below. (Greiner)

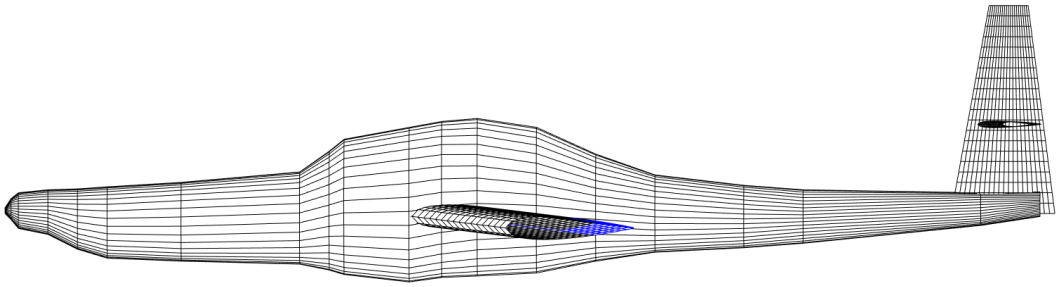


Figure 1: Aircraft Geometry Side View

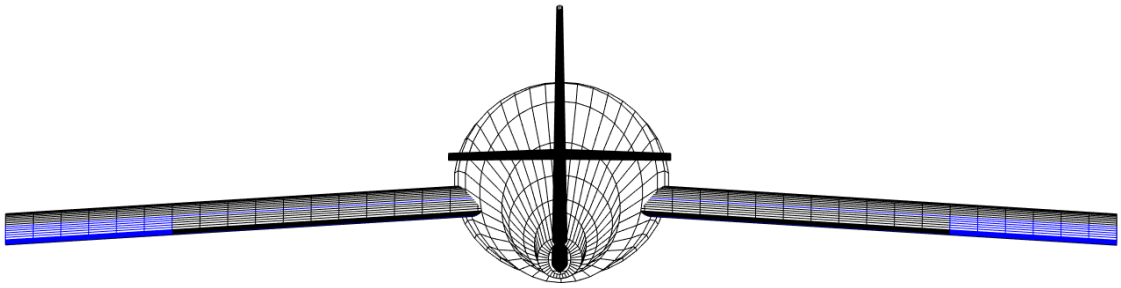


Figure 2: Aircraft Geometry Back View

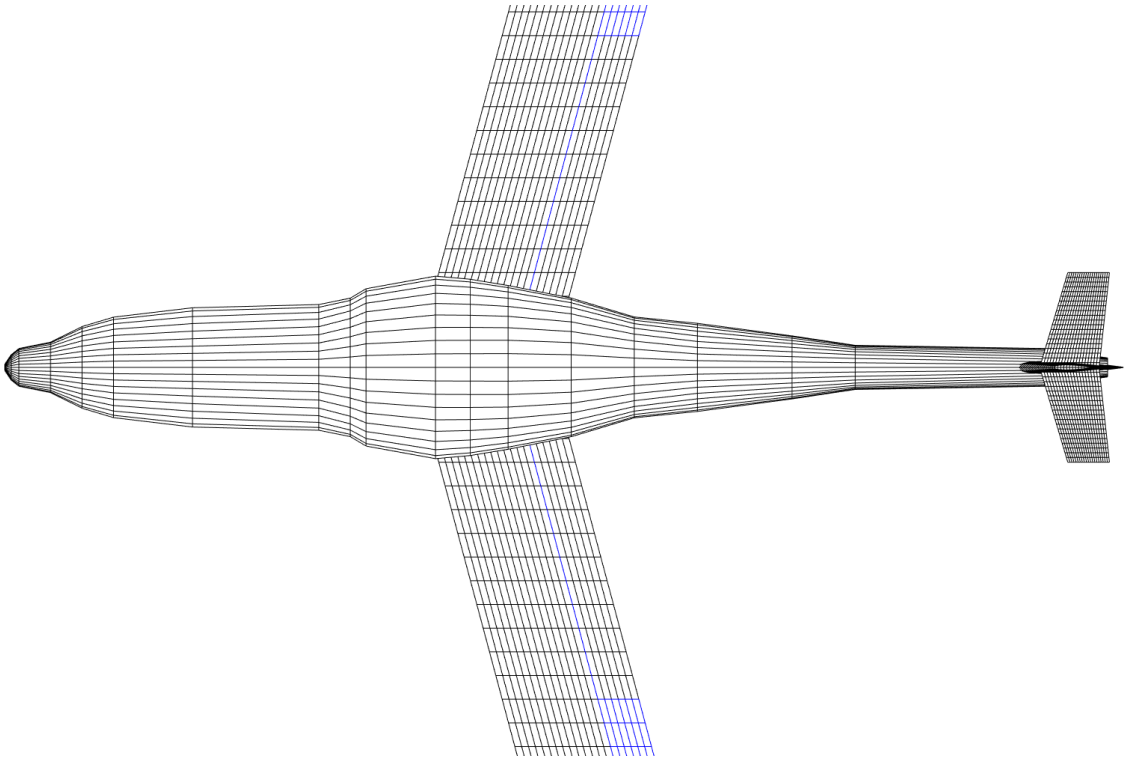


Figure 3: Aircraft Geometry Top View

Stability Derivatives

Before the aircraft is modeled with SIMULINK, calculations for stability derivatives must first take place. Calculations using empirical formulas and theory will be done in MATLAB, and they will be compared with DATCOM results. This section will indicate what empirical equations or theory was used, and how it was implemented in MATLAB. To see these equations implemented in MATLAB, see Appendix IV – MATLAB Functions for Stability and Control Derivatives. For all trimming functions and for stability calculations, conditions shown in Table 7 were used.

Table 7: Cruise Conditions Utilized for Trim

	Units	Value
M	(--)	0.525
alt	ft	6500.000

Hand Calculations with MATLAB

The first set of calculations regarding aircraft stability were performed using traditional equations as presented Dr. Moncayo for AE 413 (Moncayo). These calculations were automated using functions provided in Appendix IV – MATLAB Functions for Stability and Control Derivatives, but are presented here to reference the equations utilized.

Longitudinal Stability Derivatives

The first set of equations for longitudinal stability used originally come from incompressible aerodynamics, and thus are a good approximation for flight characteristics outside of extreme angles of pitching, where the flow regime changes during stall. These equations assume that all lift is produced by the wings, and that the horizontal tail and fuselage have no contributions to lift or drag. In order to get a better estimation, the frontal area of the fuselage could have been used achieve a more accurate C_{D_o}, but since that method was not a part of this class, the following equations should prove sufficient for an introductory stability model.

$$C_D = C_{D_o} + \frac{C_L^2}{\pi e\, AR} \quad (1)$$

$$C_{L_\alpha} = \frac{C_{l_\alpha}}{1+\frac{C_{l_\alpha}}{\pi e AR}} \quad (2)$$

$$C_{D_\alpha} = \frac{2C_L C_{L_\alpha}}{\pi e AR} \quad (3)$$

For the fuselage contribution to pitching moment, it was chosen to use a pre-provided software created by Meagan Shivers (Shivers). This MATLAB code uses the following procedure to calculate the fuselage's contribution to the pitching moment derivative. For the other contributions to the pitching moment derivative, the same downwash model was used in conjunction with another empirical formula for the C_{L_α}. This value is used in favor of the previously calculated value for the wing and tail's contributions. In the equation for $AR_{v,eff}$, the values for all inputs came from tables shown in Appendix I – Marked Table Values.

$$C_{M_{\alpha_f}} = \frac{\pi k}{2S\overline{c}} \int_0^{l_f} b_f^2 \left(1 + \frac{d\epsilon}{d\alpha}\right) dx \quad (4)$$

$$\frac{d\epsilon}{d\alpha} = 4.44 \left[K_A K_\lambda K_H \left(cos\Lambda_{\frac{c}{2}} \right)^{.5} \right]^{1.19} \quad (5)$$

$$K_A = \frac{1}{AR} - \frac{1}{1+AR^{1.7}} \quad (6) \qquad K_\lambda = \frac{10-3\lambda}{7} \quad (7) \qquad K_H = \frac{1-\frac{h_h}{b}}{\sqrt[3]{\frac{2l_h}{b}}} \quad (8)$$

$$C_{M_{\alpha_w}} = C_{L_{\alpha,w}} \overline{x_a} \quad (9)$$

$$C_{M_{\alpha_w}} = \frac{2\pi AR}{2 + \sqrt{\frac{AR\,\beta^2}{k^2}\left(1 + \frac{\tan^2 \Lambda_{\frac{c}{2}}}{\beta^2}\right)}} \overline{x_a} \quad (10)$$

$$C_{M_{\alpha_t}} = C_{L_{\alpha,t}} \left(1 - \frac{d\epsilon}{d\alpha}\right) \eta \overline{V} \quad (11)$$

$$C_{M_{\alpha,t}} = \frac{2\pi AR_{v,eff}}{2 + \sqrt{\frac{AR\,\beta^2}{k^2}\left(1 + \frac{\tan^2 \Lambda_{\frac{c}{2}}}{\beta^2}\right)}} \overline{x_a} \quad (12)$$

$$AR_{v,eff} = \left(\frac{AR_v(B)}{AR_v} AR_v\right) \left[1 + K_H \left(\frac{AR_V(HB)}{AR_v(B)} - 1\right)\right] \quad (13)$$

$$C_{M_\alpha} = C_{M_{\alpha_f}} + C_{M_{\alpha_w}} + C_{M_{\alpha_t}} \quad (14)$$

Directional Stability Derivatives

Equations for directional stability derivatives are empirical across the board, with many constants that need to be obtained from tables. K_{RI}, K_N, and k are all values obtained from charts, their values indicating at the end of Appendix I – Marked Table Values.

$$C_{N_{\beta,f}} = -K_N K_{RI}\left(\frac{S_f}{S}\right)\left(\frac{l_f}{b}\right)\left(\frac{180}{2\pi}\right) \quad (15)$$

$$C_{N_{\beta,\Gamma,\mathrm{w}}} = \frac{.75 C_L}{\Gamma} \quad (16)$$

$$C_{N_{\beta,\Lambda,\mathrm{w}}} = \frac{1}{4\pi AR} - \frac{\tan\Lambda_{\frac{c}{4}}}{\pi AR(AR+4\cos\Lambda_{\frac{c}{4}})} * \left(\cos\Lambda_{\frac{c}{4}} - \frac{AR}{2} - \frac{AR^2}{8\cos\Lambda_{\frac{c}{4}}} - \frac{6\overline{x_a}\sin\Lambda_{\frac{c}{4}}}{AR}\right) \quad (17)$$

$$C_{N_{\beta,\Lambda,\mathrm{v}}} = k\, C_{L_{\alpha,vtail}}\left(1 - \frac{d\sigma}{d\beta}\right)\eta_v \overline{V_2} \quad (18)$$

$$\left(1 - \frac{d\sigma}{d\beta}\right)\eta_v = 0.724 + \frac{3.06\left(\frac{S_v}{S}\right)}{1+\cos\Lambda_{\frac{c}{4}}} + \frac{.4 z_w}{df_{max}} + .009AR \quad (19)$$

$$C_{N_\beta} = C_{N_{\beta,f}} + C_{N_{\beta,\Lambda,\mathrm{w}}} + C_{N_{\beta,\Gamma,\mathrm{w}}} + C_{N_{\beta,\mathrm{v}}} \quad (20)$$

Lateral Stability Derivatives

The equations for lateral stability are a mix of empirical equations and theory. The first two are from strip theory and were completed in MATLAB with the help of trapezoidal numerical integration. Ten sections were used since the integrand was first order and is not highly resolution dependent. For the final equation in this section, k is the same as used for $C_{N_{\beta,\mathrm{v}}}$ in Directional Stability Derivatives.

$$C_{L_{\beta,\Gamma}} = -\frac{2\Gamma}{Sb}\int_0^{\frac{b}{2}} c(y)\, C_{L_\alpha} y\, dy \quad (21)$$

$$C_{L_{\beta,\Lambda}} = -\left(\frac{2\alpha\cos\Lambda\sin\Lambda}{Sb}\right)\int_0^{\frac{b}{2}\sec\Lambda} c(y)\, C_{L_\alpha} y\, dy \quad (22)$$

$$C_{L_{\beta,\mathrm{v}}} = -kC_{L_{\alpha,v}}\left(1 - \frac{d\sigma}{d\beta}\right)\eta_v \frac{S_v}{S}\left(\frac{z_v\cos\alpha - l_v\sin\alpha}{b}\right) \quad (23)$$

Control Derivatives

Control derivatives were found using theory that involved a control surface effectiveness parameter, τ_a. Although this could have been found using the table, a function representing the table was created in MATLAB to simulate this relationship between the surface area of the control surface and the lifting surface. As a result, all the below equations could be calculated without any lookup table, based on previous values. Note that for the C_{l_α} values for the vertical and horizontal tail it was approximately set to that of the wing. This is likely accurate for the horizontal tail but may not be so for the vertical tail since it is not cambered or set at incidence. To note is that the constant k for the rudder's derivative is the same as previously used for $C_{L_{\beta,\mathrm{v}}}$ and $C_{N_{\beta,\mathrm{v}}}$ in Directional Stability Derivatives and Lateral Stability Derivatives sections, respectively.

$$C_{L_{\delta a}} = \frac{2C_{l_\alpha}\tau_a}{Sb}\int_{y_l}^{y_u} c_a(y)y\,dy \quad (24)$$

$$C_{N_{\delta r}} = -k\eta_v \frac{l_v S_v}{\bar{c}S} C_{L_{\alpha,v}}\tau_v \quad (25)$$

$$C_{M_{\delta e}} = -\frac{l_h S_h}{\bar{c}S}\eta_h C_{L_{\alpha,h}}\tau_e \quad (26)$$

DATCOM Results

In order to compare hand-calculations with a software derivative calculator such as DATCOM, first the aircraft must be modeled with appropriate resolution. It was chosen to use twenty sections from nose to tail, which was the maximum that the version of digital DATCOM provided allowed for. The fuselage, wings, horizontal tail, and vertical tail were all modeled for with the help of the plotting script provided by Embry-Riddle (Greiner). Figure 4 shows a representation of this model.

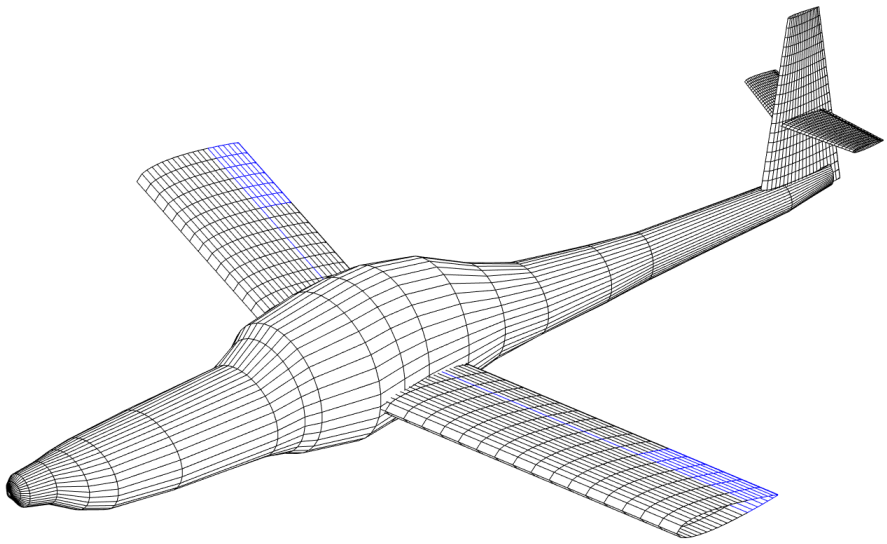


Figure 4: DATCOM Representation of Aircraft

This model was run in (Digital Datcom Program), with an input file shown inAppendix II – DATCOM Input File. These outputs were then transferred into .mat files in MATLAB and prepared for integration into SIMULINK. It is important to note here that it was chosen to input a wide range of Machs and angles of attack into Digital Datcom to allow for stall characteristics to be seen when simulating the aircraft with SIMULINK and FlightGear. An example of this can be seen below in Figure 5, where stall characteristics were only seen at most Mach numbers above 20deg in DATCOM. Although DATCOM does have limitations that make it only valid for small angles of attack, this method was chosen to allow for a more realistic simulation.

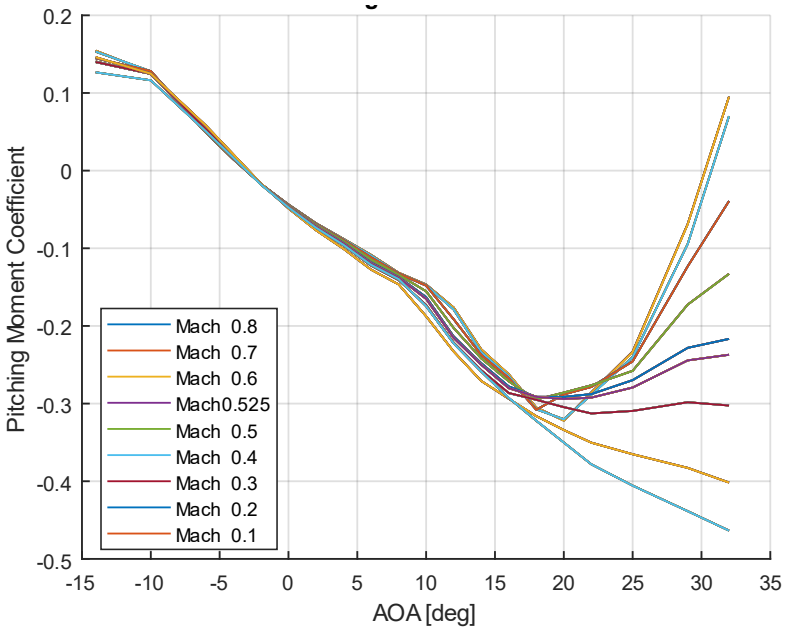


Figure 5: DATCOM Pitching Moment Coefficient vs Angle of Attack Showing Stall Characteristics

Stability Derivative Comparison – DATCOM vs MATLAB

In order to check the validity of both methods of calculating stability derivatives, their results were compared in Table 8. This was done at a Mach of .5, an altitude of 5,000ft and zero angle of attack.

Table 8: Stability Derivative Comparison

	Units	DATCOM	MATLAB	% DIFF
C_M_alpha	1/rad	-0.7951	-0.3109	87.56%
C_N_beta	1/rad	0.0959	0.1688	55.08%
C_L_beta	1/rad	-0.0140	-0.0301	73.08%

Although these results are not entirely comparable, since the empirical and theoretical equations used are a different method to that of DATACOM, these differences were determined to be acceptable. What is important is that both methods yielded a result within the same order of magnitude and have the same sign.

SIMULINK Integration

In order to properly model the aircraft in SIMULINK, a process of modifying a pre-existing model of the SIAI-Marchetti S-211 was performed. This was done due to the similarities of the aircraft allowing an iterative integration to occur. A mixture of hand-calculated control derivatives and DATCOM stability derivatives were input into the model to allow for a thorough analysis of the designed plane to occur. Afterwards, the model was integrated into FlightGear, which allowed the aircraft to be tested in real time, allowing for real pilots to test the aircraft and give feedback on the control characteristics of the design.

Lookup Tables

Once the stability derivatives were complete, it was time to model the aircraft in a simulation environment. Using built in MATLAB functions, the output from Digital Datcom was saved as matrices, with values for a range of altitudes, Machs, and angles of attack. These were then implemented as lookup tables in the Simulink model, examples shown in the following figures. Since the stability derivatives were automatically pulled from Datcom files and imported into the workspace, it was chosen to implement all derivatives possible; this meant $C_{N_\beta}, C_{N_p}, C_{N_r}, C_{M_\alpha}, C_{M_q}, C_{L_\alpha}, C_{L_q}, C_{l_p}, C_{l_r}, C_{l_\beta}, C_{Y_\beta}$, and C_{Y_p}. These implementations can be found in the following figures.

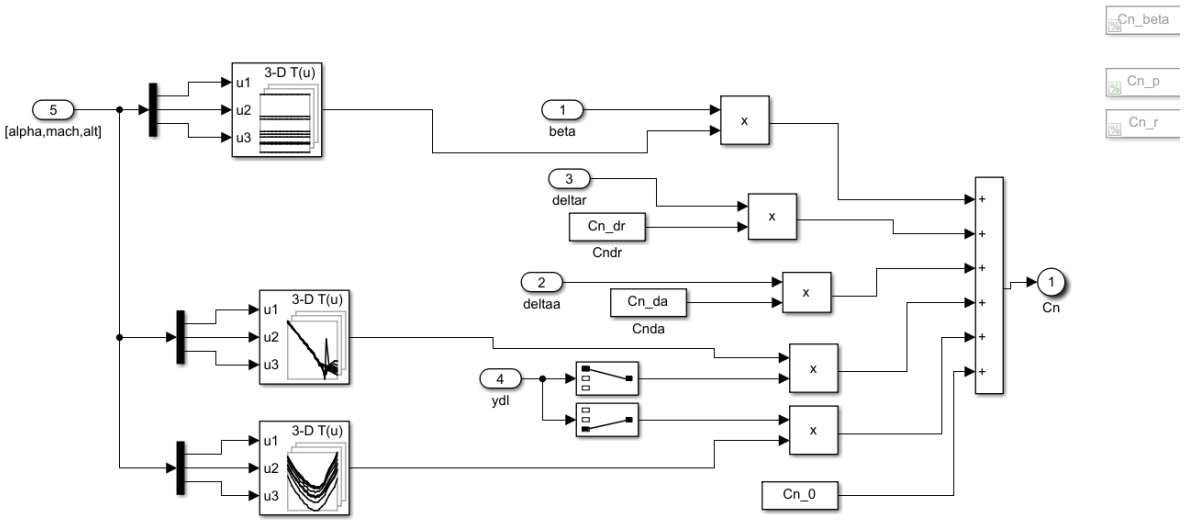


Figure 6: C_N Lookup Implementation

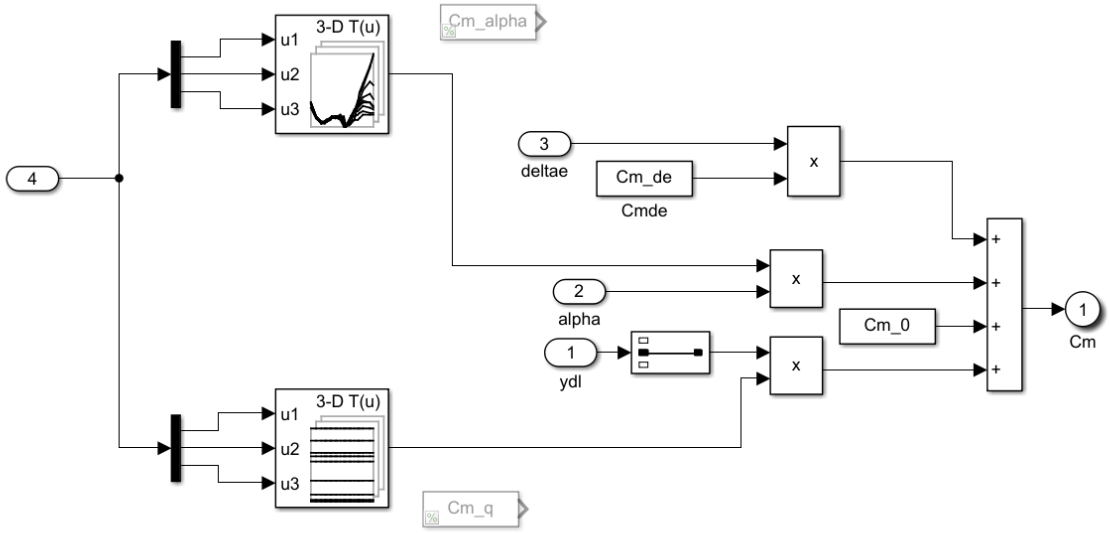


Figure 7: C_M Lookup Implementation

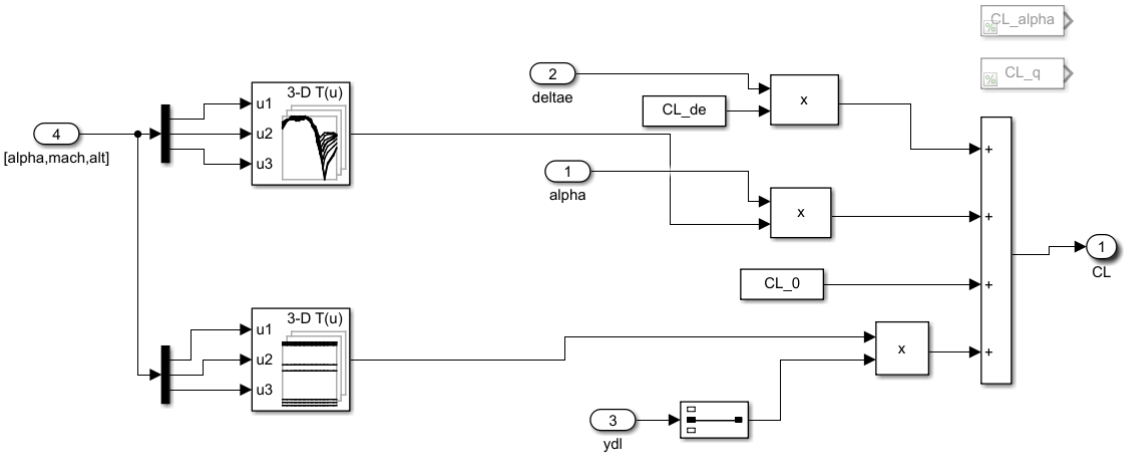


Figure 8: C_L Lookup Implementation

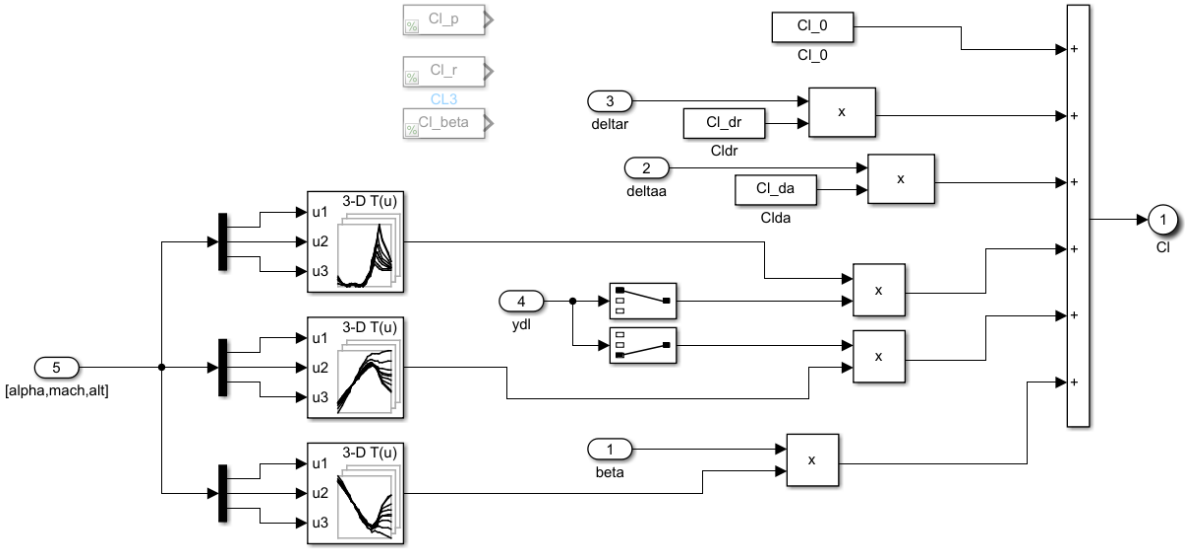


Figure 9: C_l Lookup Implementation

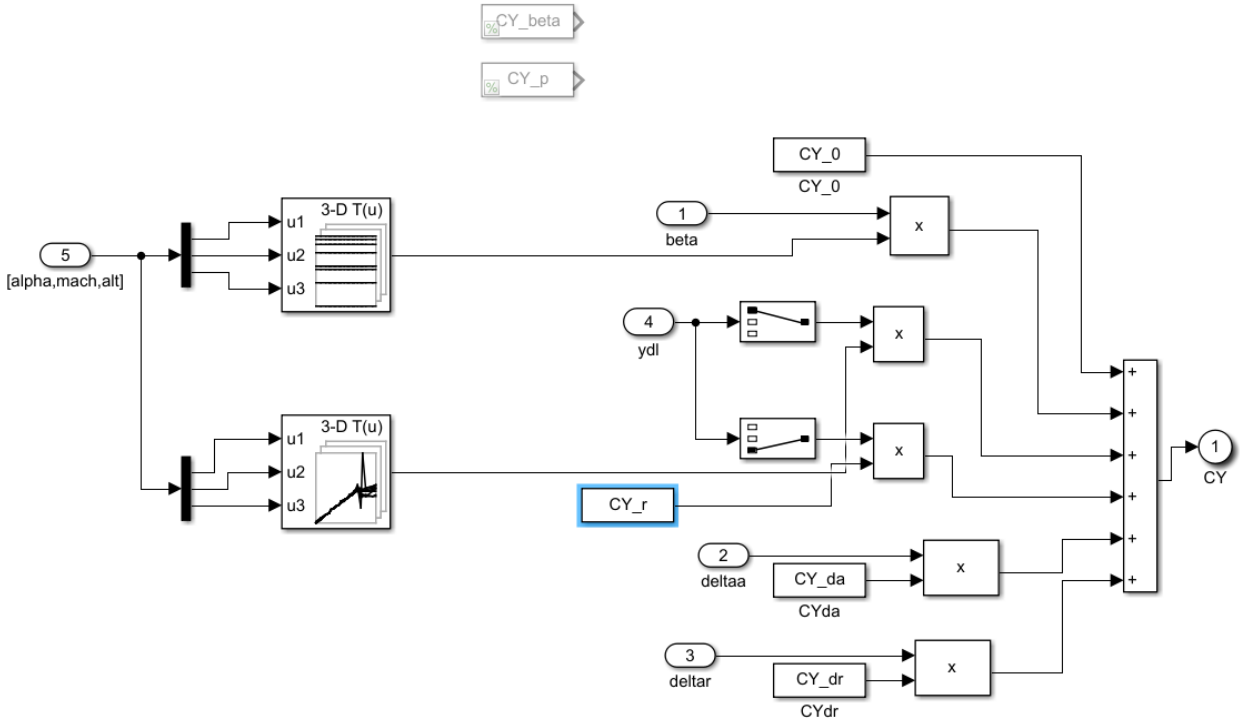


Figure 10: C_Y Lookup Implementation

FlightGear Interface

In order to implement an interface to FlightGear, the pre-existing Simulink model was modified slightly to accommodate sending packets via internal IP addresses to FlightGear, which was capable of displaying the aircraft. This was done with a .bat script that started a FlightGear instance, looking for input regarding positioning and aircraft information from an IP address. Then, a modified Simulink model was run that sent the packets FlightGear was expecting, shown in Figure 11.

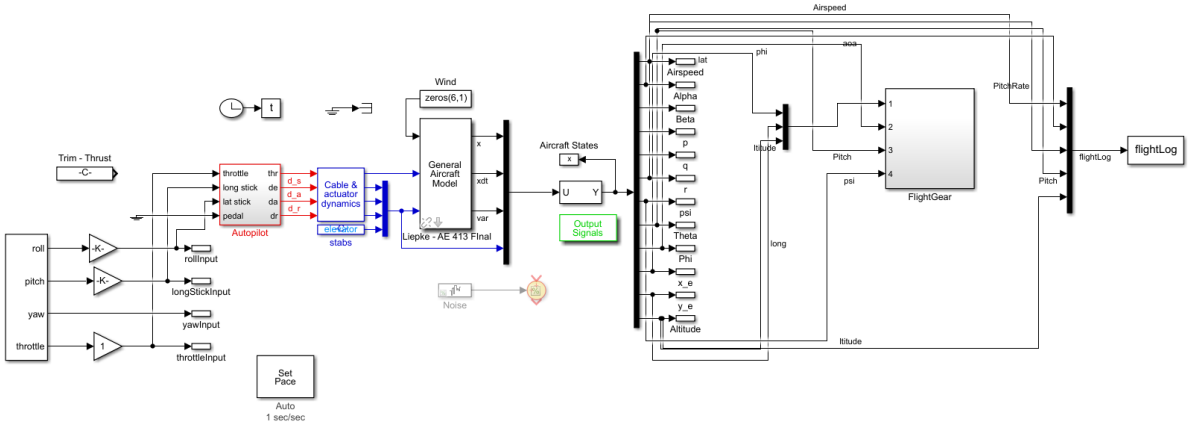


Figure 11: Complete Simulink Simulation Front Page with FlightGear Interface

Aircraft Performance – Simulation Results

In order to model the aircraft's response to user input, the simulated aircraft was subjected to varying excitations that were meant to bring the aircraft out of equilibrium. These tests were performed to the trimmed model at Mach .525, and included excitation of each control direction independently: elevator, aileron, and rudder. Each excitation included 2seconds of negative input followed by two seconds of positive input via a step function. The following 100 seconds were recorded and the data is shown in the following sections. The aircraft was also tested by certified pilots, who provided feedback on the aircraft.

Trim Characteristics

Due to the nature of the tests that are about to be performed, it was chosen to run an initial test run with the trimmed aircraft. This will act as a control for the excitation tests that were conducted. It is important to note that with no excitation there is a low frequency and high frequency oscillation in pitch; the high frequency oscillation dampens after a few seconds, while the low frequency oscillation does not, albeit small in magnitude

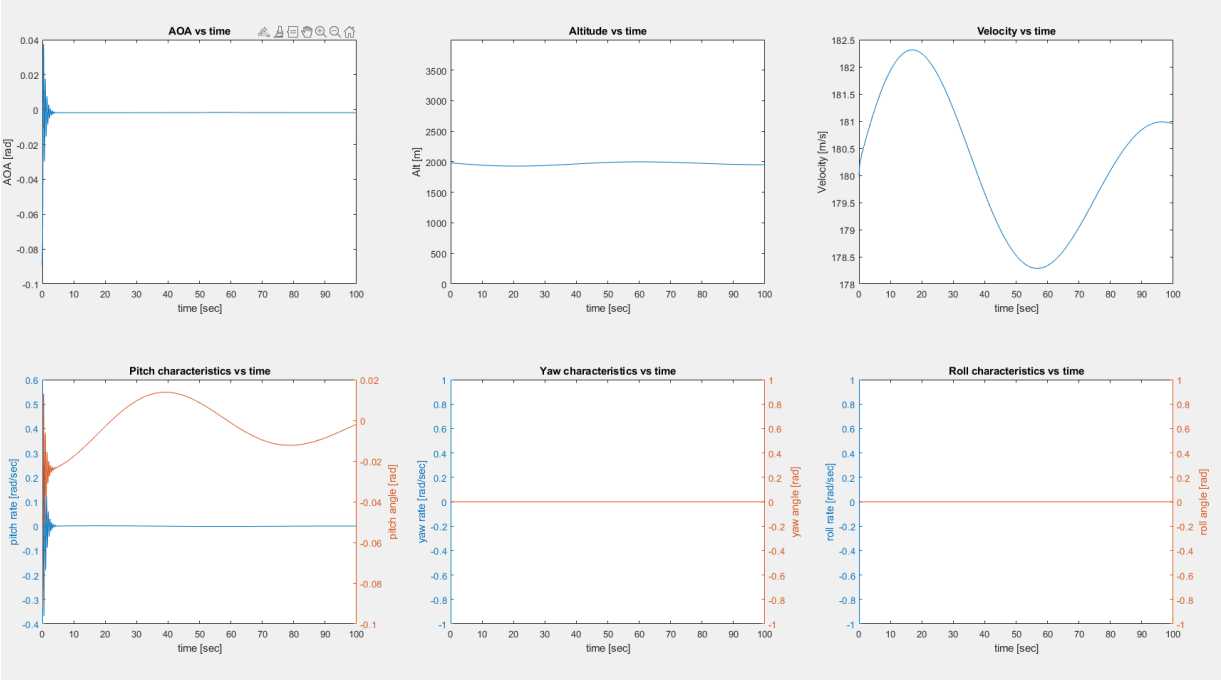


Figure 12: Trim Response

Elevator Excitation

With Elevator Excitation there is little unexpected behavior. The aircraft holds altitude and quickly dampens to trim flight. This process takes no longer than 20 seconds and can be considered a good result. As expected with a sharp change in pitch caused by this elevator excitation, the aircraft's velocity changes quickly, although it returns to trim conditions quickly.

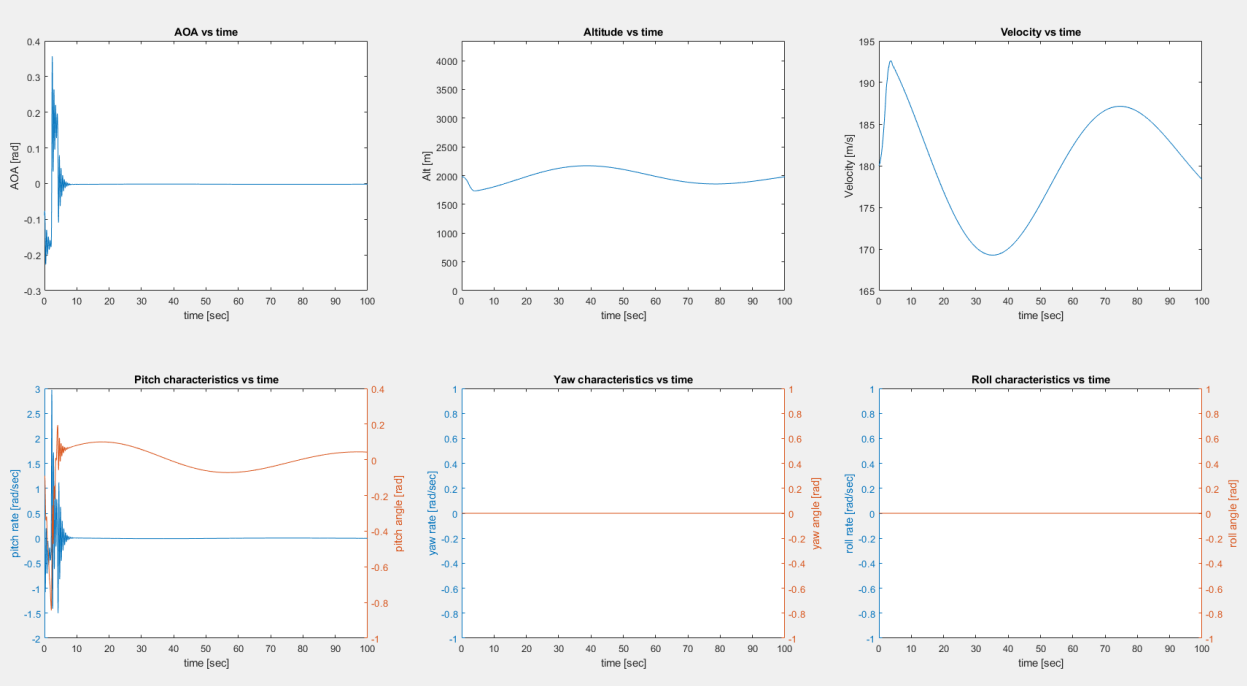


Figure 13: Elevator Excitation Response

Aileron Excitation

With aileron excitation all characteristics of the aircraft come back to an equilibrium state relatively quickly. Looking at the magnitude of pitch, yaw, and roll, although still oscillatory near the end of the solution time, nothing is out of an acceptable range. It is clear that the yaw may require more dampening, which may be an indication of what is to come in the rudder test.

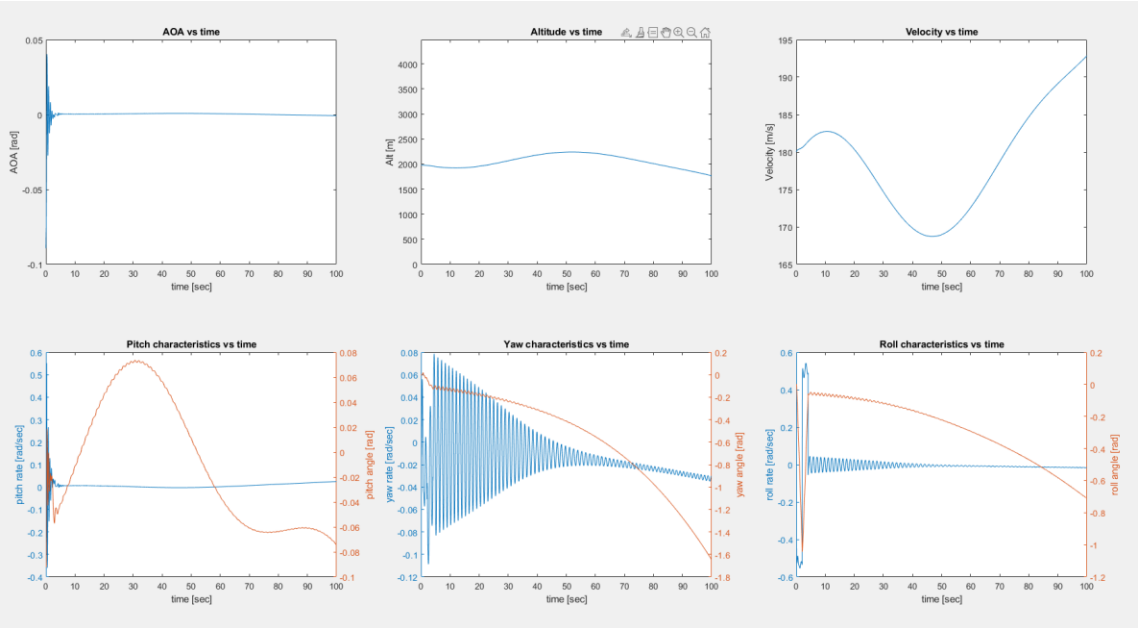


Figure 14: Aileron Excitation Response

Rudder Excitation

Rudder excitation bring more concern to the underdampened yaw characteristics of the aircraft. Although trending towards an equilibrium position, the yaw of the aircraft would be extremely uncomfortable and make the pilot nauseous. This response is the most alarming out of all of the tests, and should be remedied if the aircraft is to undergo further design.

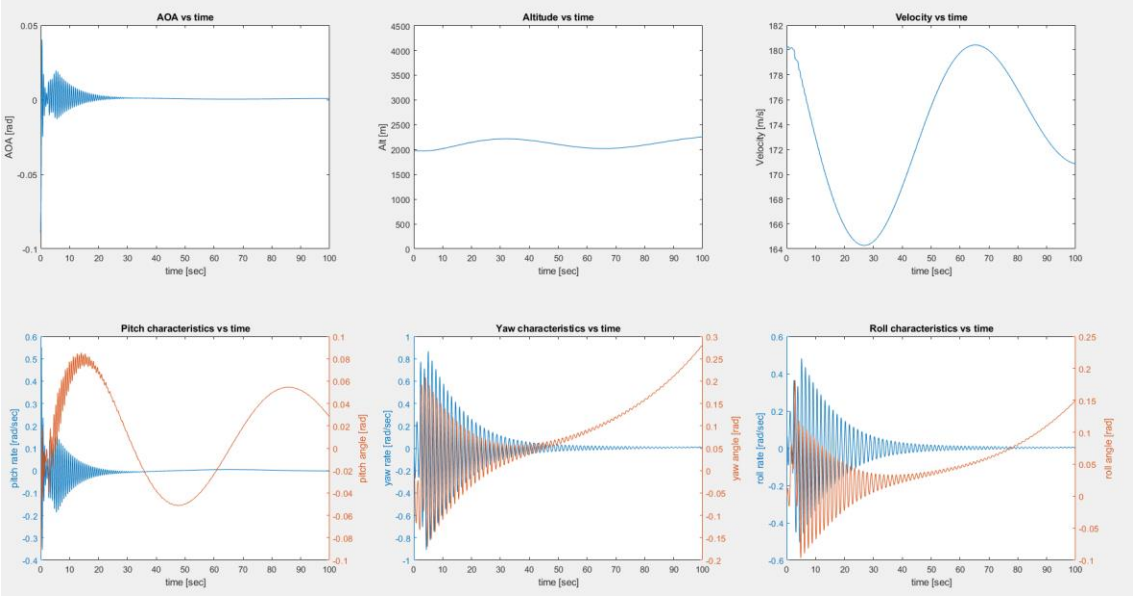


Figure 15: Rudder Excitation Response

Pilot Feedback

With commercial pilot available to test the aircraft in the FlightGear interface, he was introduced to the simulation environment with all lookup tables implemented. The pilot was given a joystick and thrust lever; a rudder solution was not available to the pilot. It is also notable that although the FlightGear controls worked well, there was no way to see airspeed in the simulated cockpit, meaning the pilot did not have the sufficient information to control the aircraft as he would have desired.

Feedback from the commercial pilot was mixed. He indicated that "it is a bit twitchy," referring to the pitching input provided. He noted that although he expected a trainer to be more responsive and touchier than a GA aircraft, he felt like the controls were too particular. He also noted that the oscillatory yawing of the aircraft when any directional input was given would be "terrible." Data from his flight can be seen below.

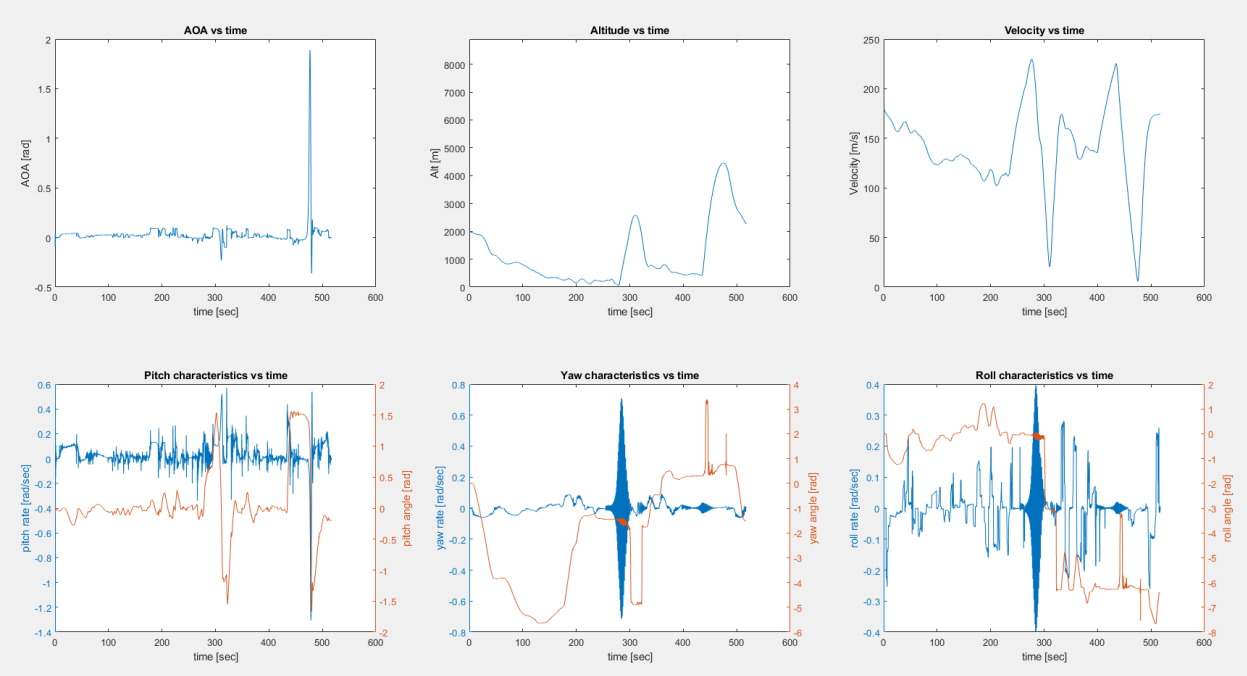


Figure 16: Commercial Pilot FlightGear Test Flight

Afterwards the engineer tested the controls with FlightGear and noted the same things that the pilot did. When the airspeed was too high there was significant oscillations in yaw that transferred to roll when banked. This could be a side effect of the lookup tables, or it could be due to the poor directional stability.

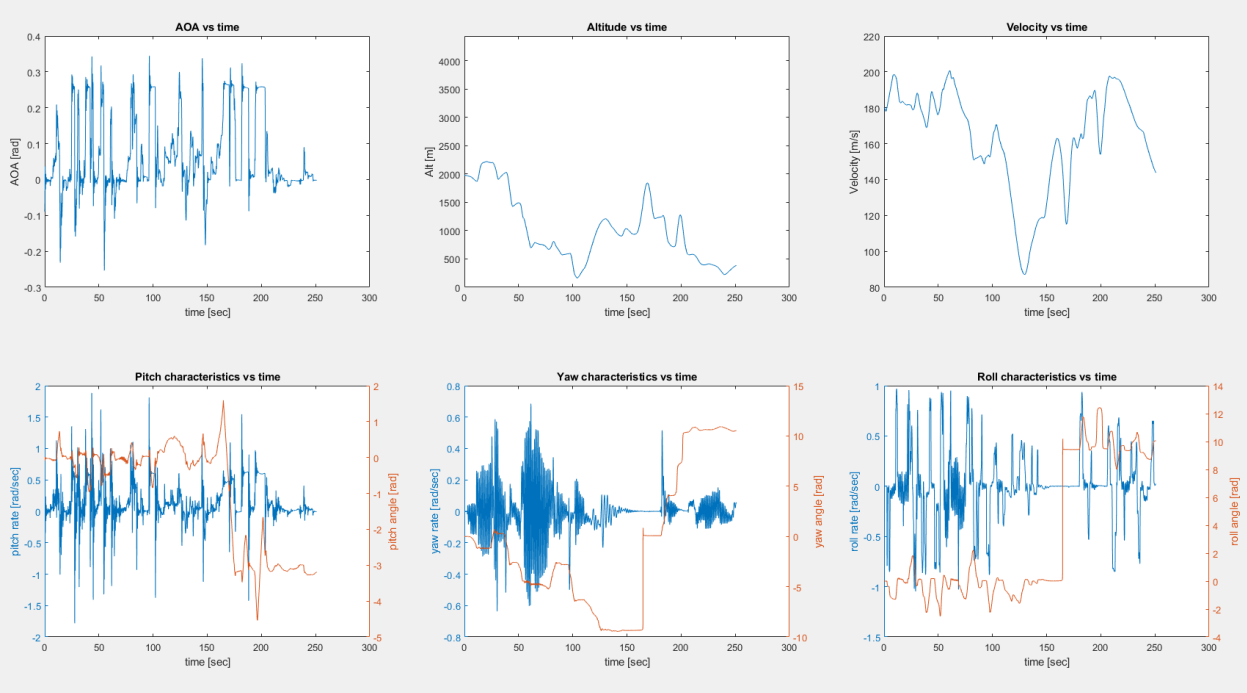


Figure 17: Engineer FlightGear Test Flight

Conclusion

After going through the process of calculating and implementing stability and control derivatives in Simulink, it has become apparent that there are some issues relating to the initial design of the aircraft. Primarily, the yaw oscillations need to be dealt with if the designed aircraft would ever be built and flown. It is important to note that although every characteristic of the aircraft satisfied requirements for Test 1, the design was not stable due to things not originally calculated or tested during the period of the test.

As far as integrating calculated derivatives into Simulink to obtain a realistic model of the aircraft, the process was a success. All possible stability derivatives were transferred from Datcom output and input into a Simulink model. The model was then successfully integrated into FlightGear, allowing for the aircraft model to be flown and tested by engineers and pilots. The files for the Simulink model should be provided along with the FlightGear '.bat' file in addition to this report.

References

Airfoil Tools. 2021. <airfoiltools.com>.

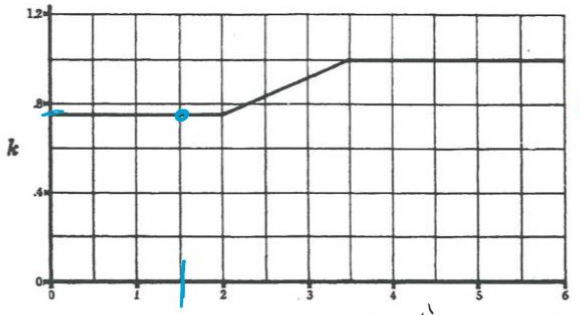
Digital Datcom Program. Public Domain Aeronautical Software, n.d.

Greiner, Glenn P, Mohammed, Jafar. *Datcom Wireframe Plot Script*. Embry-Riddle Aeronautical University. 4 March 2008.

Moncayo, Hever. *AE 413 Lecture Notes*. 2021.

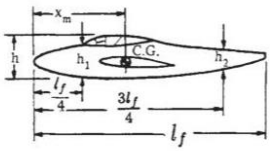
Shivers, Maegan. *Matlab Multhopp Code*. n.d. .m.

Appendix I – Marked Table Values



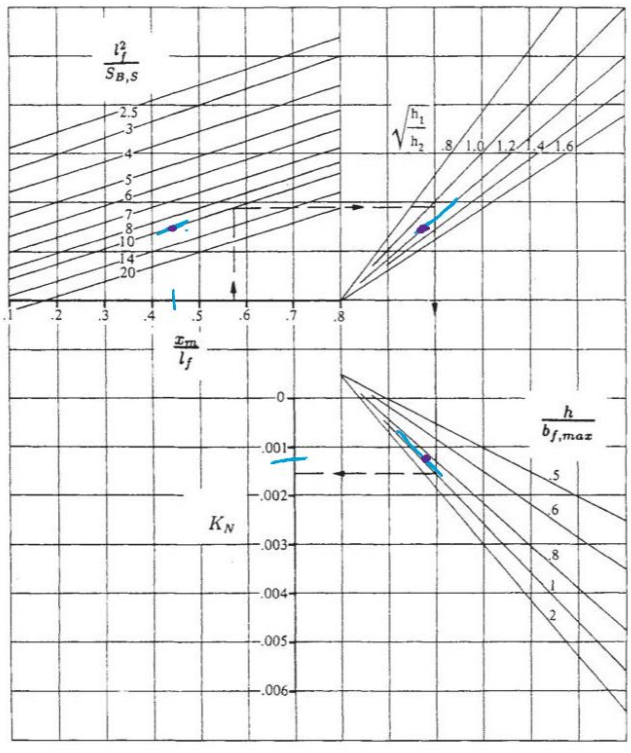
$\therefore k = .78$

$\frac{b_V}{2r_1}$ $b_v = 6'$ $2(r_1) = 2(2') = 4'$ $\rightarrow \frac{b_v}{2r_1} = 1.5$



$S_{B,S}$: Body Side Area

$b_{f,max}$: Maximum Body Width



$\frac{x_m}{l_f} = \frac{11'}{35'} = .45$

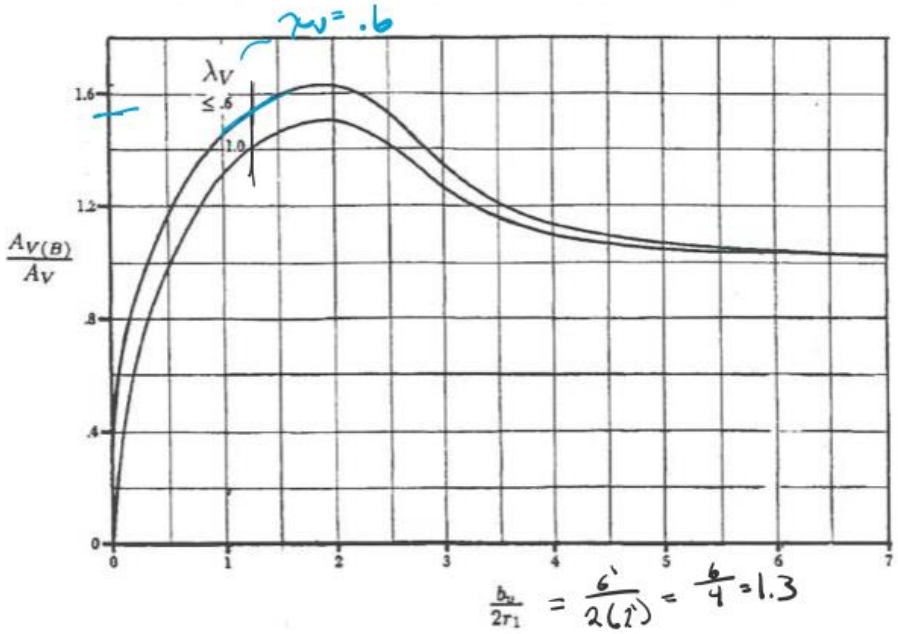
$\frac{l_f^2}{S_{B,S}} = \frac{35^2}{120} = 10.2$

$\sqrt{\frac{h_1}{h_2}} = \sqrt{\frac{2.7}{1.8}} = 1.22$

$\frac{h}{b_{f,max}} = \frac{5.5}{6} = .9$

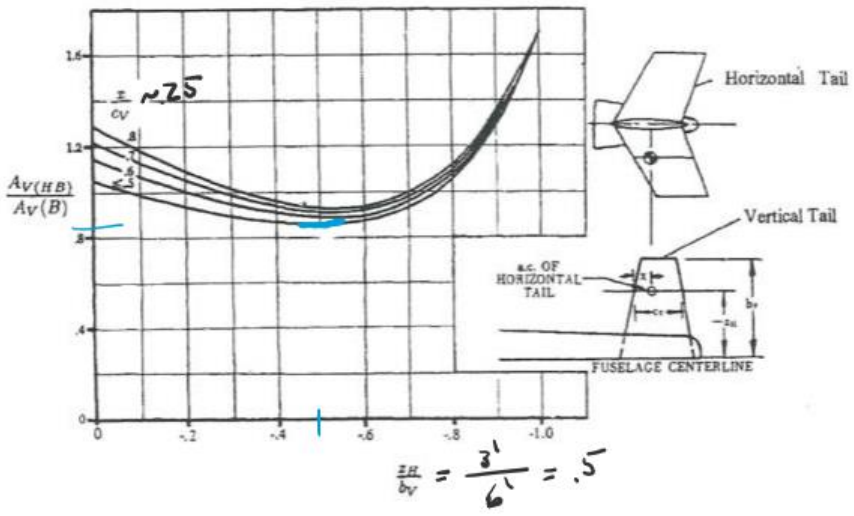
$K_n = .0012$

Fig. 3.73 Empirical factor K_N related to $(C_{n\beta})_{B(W)}$ (Ref. 1).



$\frac{A_{V(B)}}{A_V} = 1.55$

Fig. 3.77 Empirical parameter $A_{V(B)}/A_V$ as a function of $b_V/2r_1$ (Ref. 1).



$\frac{A_{V(HB)}}{A_{V(B)}} = 0.82$

Fig. 3.78 Empirical parameter $A_{V(HB)}/A_{V(B)}$ as a function of z_H/b_V (Ref. 1).

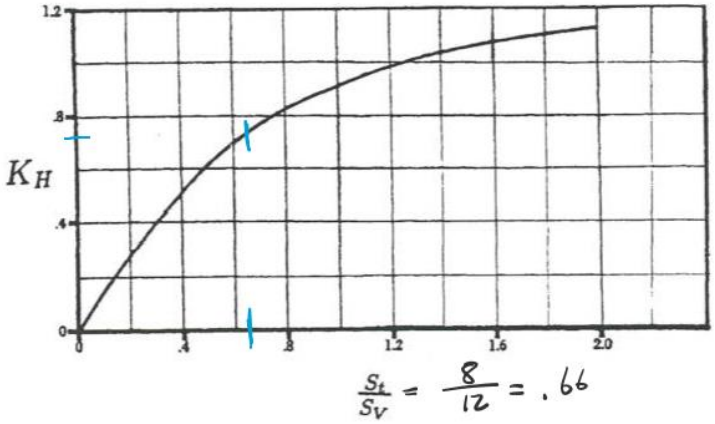


Fig. 3.79 Empirical parameter K_H as a function S_t/S_V (Ref. 1).

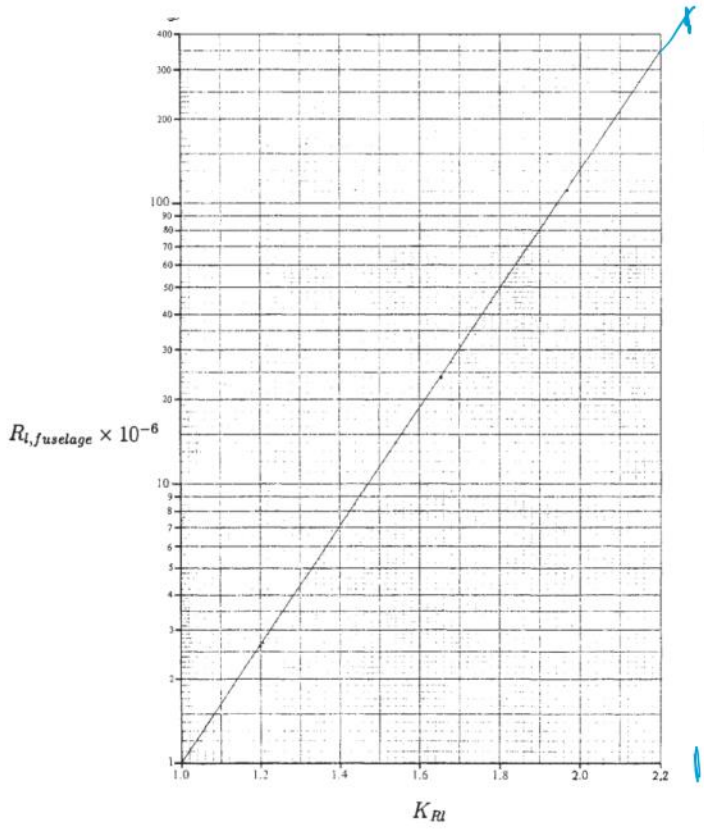


Fig. 3.74 Variation of K_{Rl} with fuselage Reynolds number.[1]

$$AR_{v,eff} = \left(\frac{AR_v(B)}{AR_v} AR_v\right)\left[1 + K_H\left(\frac{AR_v(HB)}{AR_v(B)} - 1\right)\right]$$

$$AR_{v,eff} = (1.55(2.4))(1 + .75(.82 - 1)) \;=\; 3.2$$

SUMMARY:
K_Rl = 2.3
K_N = .0012
K = .78
$\boldsymbol{AR_{v,eff} = 3.2}$

Appendix II – MATLAB Output

This can be recreated by running 'SoftwareDeliverables/MatlabHandCalculations/calculateDerivativesMain.m' and inputting the variables at the top of the file in order.

```
Contribution to C_M_alpha:
      Fuselage : 0.03543
      Wing : 0.59097
      Tail : -0.93729
K_RI via Fig 3.74?2.3
K_N via Fig 3.74?.0012
k from Fig 3.75 for vtail? should be around 1: .78
AR_effective of the tail from 3.77, 7.78, and 3.79? should be around
AR_vtail: 3.2
Contribution to C_N_beta:
      Fuselage : -0.17030
      Wing Dihedral : -0.00901
      Wing Sweep : 0.02146
      Vertical Tail : 0.28758
C_L_B Contributions:
      Dihedral:   0.0008
      Sweep:      -0.0004
      Tail: -0.0006

************LATERAL STABILITY DERIVATIVE RESULTS**********
Airfoil Cl_0:     0.114700    Required Cruise Cl:     0.120139

C_D @ cruise:     0.004634

C_L_alpha:  5.126560 [1/rad]
      Required: 4 - 6 [1/rad]

C_D_alpha:  0.058388 [1/rad]

C_M_alpha, total: -0.3109 [1/rad]
      Required -.35 to -0.15


************CRUISE CHARACTERISTICS RESULTS**********
L/D @ cruise:     25.924808

Lift @ cruise:    5000 [lbf]

      For reference, weight is:     5000 [lbf]

Drag @ cruise:    193 [lbf]

      For reference, max thrust is: 3371 [lbf]

To Cruise, throttle trim is about:  5.722 %
To Cruise, wing pitch must be about:      3.483 [deg]
```

```
************DIRECTIONAL STABILITY DERIVATIVE RESULTS**********
C_N_beta, total:  0.1297 [1/rad]
      Required .02 to .3

C_L_beta, total:        -0.0002 [1/rad]

************CONTROL DERIVATIVE RESULTS**********
C_L_dAileron :          0.00224 [1/rad]
C_N_dRudder :           -0.00368 [1/rad]
C_M_dElevator :         -0.02552 [1/rad]

************CONTROL DERIVATIVE RESULTS - DEG**********
C_L_dAileron :          0.12858 [1/ded]
C_N_dRudder :           -0.21091 [1/deg]
C_M_dElevator :         -1.46244 [1/deg]

************RANDOM OTHER THINGS**********
Stall @ sea level:      157.7 [ft/s]

      For reference, cruise is M = 0.53 or v = 572.2 [ft/s]

The Neutral Point is 0.0202 [ft] behind the AC

Empirical Downwash is:  0.254

c_bar for the wings:    4.3 [ft]
            Required 4 to 6 [ft]
```

Appendix II – DATCOM Input File

This can also be found at 'SoftwareDeliverables/Simulink/liepke.dat'

```
CASEID LIEPKE AE413 TEST1 OR HW7
 $FLTCON NMACH=9.0,MACH(1)=0.1,0.2,0.3,0.4,0.5,0.525,0.6,0.7,0.8$
 $FLTCON NALT=10.0,ALT(1)=0.0,1000.0,2000.0,3000.0,4000.0,5000.0,6000.0,8000.0,
 10000.0,20000.0$
 $FLTCON NALPHA=20.0,ALSCHD=-14.0,-10.0,-6.0,-4.0,-2.0,0.0,2.0,4.0,6.0,
 8.0,10.0,12.0,14.0,16.0,18.0,20.0,22.0,25.0,29.0,32.0,
 GAMMA=0.0,WT=5000.0,STMACH=0.9,TSMACH=1.3,LOOP=2.0$
 $OPTINS SREF=180.0,CBARR=5.67,BLREF=33.38$
 $SYNTHS XCG=15.0,ZCG=0.0,XW=13.0,ZW=0.0,ALIW=0.0,XH=32.85,
  ZH=3.0,ALIH=0.0,XV=32.5,ZV=0.0,XVF=32.5,ZVF=3.0,VERTUP=.TRUE.$
 $BODY NX=20.0,ITYPE=1.0,
  X=.05,.25,.5,1.5,2.5,3.5,6.0,10.0,11.0,11.5,13.7,14.8,16.0,18.0,20.0,
  22.0,25.0,27.0,33.0,35.0,
  ZU=.2,.5,.7,.8,.83,.9,1.05,1.4,2.1,2.5,2.9,3.1,3.2,2.9,2.0,1.3,.95,
  .8,.7,.74,
  ZL=0.0,-.2,-.5,-.7,-1.2,-1.5,-1.6,-1.7,-1.9,-2.0,-2.21,-2.2,-2.23,-2.2,
  -2.0,-1.7,-1.2,-1.0,-.7,-.5,
  S=0.0314,0.439,1.130,1.884,4.145,6.031,7.908,9.738,13.823,17.671,23.277,
  23.310,22.176,17.624,10.053,6.597,3.377,1.979,1.319,0.973,
  P=1.088,3.998,6.529,8.347,12.168,14.654,16.767,18.644,22.514,25.425,
  29.110,29.393,29.135,26.542,20.4179,15.969,11.433,9.126,7.273,6.341,
  R=.1,.4,.6,.8,1.3,1.6,1.9,2.0,2.2,2.5,2.9,2.8,2.6,2.2,1.6,1.4,1.0,
  .7,.6,.5$
NACA-W-4-1408
 $WGPLNF CHRDR=4.33,CHRDTP=4.33,SSPN=15.0,SSPNE=12.7,
  SAVSI=15.0,CHSTAT=0.25,TWISTA=0.0,DHDADI=-2.0,TYPE=1.0$
NACA-H-4-1408
 $HTPLNF CHRDR=1.88,CHRDTP=1.32,SSPN=3.0,SSPNE=3.0,
  SAVSI=14.0,CHSTAT=0.25,TWISTA=3.5,DHDADI=0.00,TYPE=1.0$
NACA-V-4-0008
 $VTPLNF CHRDR=3.5,CHRDTP=1.317,SSPN=7.0,SSPNE=7.0,
  SAVSI=6.0,CHSTAT=0.25,TYPE=1.0$
```

DAMP

DERIV RADIANS

NEXT CASENEXT CASE

Appendix III – Simulink Code

See ‘SoftwareDeliverables/Simulink/LiepkePlane.slx’

Appendix IV – MATLAB Functions for Stability and Control Derivatives

Intro

Matthew Liepke, AE 413 Spring 2021

This script serves as a publishing tool to show all work done to calculate stability and control derivatives for Test 1, miscellaneous Homeworks and the Final Project.

Contents

Static Longitudinal Stability Derivatives

```
function C_L_alphawing = C_L_alphawing(AR, M, K, sweep_half)
%C_L_ALPHAWING gives C_L_alpha of the wing for calculation in total C_M_alpha.
%              can be used for tail calculation if proper A, sweep_half given
% Built by Matt Liepke for AE 413 Test1
%   AR  : aspect ratio of wing
%   M   : Mach at flight
%   K   : empirical value.  can be assumed 1 unless accuracy is paramount
%   sweep_half: sweep at half the chord of the wing in radians
beta = sqrt(1-M^2);
C_L_alphawing = 2*pi*AR / (2 + sqrt((AR*beta/K)^2*(1+tan(sweep_half)^2/ (beta^2)) +
4));
end


function C_L = C_L(weight, q_inf, S)
%LIFTCOEFF Provides C_L given steady flight with aircraft parameters
% Built by Matt Liepke for AE 413 Test1
C_L = weight / (q_inf * S);
end


function C_D = C_D(AR, e, C_L, C_D0)
%COEFFDRAG Gives C_d for a given aircraft give
% Built by Matt Liepke for AE 413 Test1
%   AR  : aspect ratio
%   e   : oswald effeciency factor
%   C_L : Lift coeff
%   C_D0: Drag @ C_L = 0

C_D = C_D0 + C_L^2 /(pi*e*AR);
end


function C_L_alpha = C_L_alpha(C_l_alpha_wing, AR, e)
%LIFTCOEFF_ALPHA returns derivative of CL w/r to angle of attack in 1/rad
% Built by Matt Liepke for AE 413 Test1
%   C_l_alpha   : sectional (airfoil) C_l_alpha in 1/deg
%   AR  : Aspect Ratio
%   e   : Oswald's efficiency factor
C_L_alpha = C_l_alpha_wing / (1 + (57.3*C_l_alpha_wing)/(pi*e*AR));
end


function C_D_alpha = C_D_alpha(C_L, C_L_alpha, e, AR)
```

```
%C_D_ALPHA returns C_D_alpha given aircraft flight parameters. is the
%                    derivative of C_D w/r to alpha
% Built by Matt Liepke for AE 413 Test1
%   C_L : Lift coeff
%   e   : Oswald's efficiency factor
%   AR  : Aspect Ratio
%   C_L_alpha  : C_L_alpha for the wing (not section)

C_D_alpha = 2 * C_L * C_L_alpha / (pi*e*AR);
end


function C_M_alpha = C_M_alpha(AR, AR_tail, M, sweep_quarter, sweep_half,
sweep_half_tail, nabla, V, x_a_bar, lf, lf_up, l_h, h_h, c_r, c_t, bf, b, printoutput)
%C_M_ALPHA returns total C_M_alpha from fuselage, tail and wings
% Built by Matt Liepke for AE 413 Test1
%   AR      : aspect ratio of wing
%   AR_tail : horz tail aspect ratio
%   M       : Mach at flight
%   sweep_quarter: quarter sweep of wing
%   sweep_half: half sweep of wing
%   sweep_half_tail: half sweep of horz tail;
%   nabla   : ratio of dynamic pressure, tail/wing
%   V       : volumetric ratio of tail/wing
%   x_a_bar : normalized dist from wing AC -> CG divided by chord
%   lf      : length of fuselage in ft
%   lf_up   : length of fuselage in upwash in ft
%   l_h     : length from wing AC to tail AC
%   h_h     : vert height of horz tail from wing
%   c_r     : root chord of wing
%   c_t     : tip chord of wing
%   bf      : fuselage width in inches(array of 5 sections: [ a b c d e])
%   b       : span of wing

K = 1;%all the calculations in class had K == 1 or very close
%fuselageCont2 = C_M_alpha_fuselage_liepke(lf, lf_up, l_h, h_h, c_r, c_t, bf, b,
sweep_quarter);
fuselageCont = C_M_alpha_fuselage_shivers(lf, lf_up, l_h, h_h, c_r, c_t, bf, b,
sweep_quarter);
wingCont = C_L_alphawing(AR, M, K, sweep_half) * x_a_bar;
tailCont = -C_L_alphawing(AR_tail, M, K, sweep_half_tail) * nabla * V;

if(printoutput)
    fprintf("Contribution to C_M_alpha:\n\tFuselage : %.5f\n\tWing : %.5f\n\tTail :
%.5f\n",fuselageCont, wingCont, tailCont);
end
C_M_alpha = fuselageCont + wingCont + tailCont;
end
```

Static Directional Stability Derivatives

```
function [C_N_beta, k_vtail] = C_N_beta(C_L, Gamma, AR, M, sweep_quarter,
sweep_quarter_vtail, sweep_half_vtail,x_a_bar, S_fuselage, S,S_v, b, df_max, z_w,
l_f,l_v, Re_c, c_bar)
%C_N_BETA Returns total C_N_beta for an aircraft given following parameters:
% Built by Matt Liepke for AE 413 Test1
%   C_L     : C_L of the wing
%   Gamma   : wing dihedral in radians
%   AR      : Aspect ratio of wings
%   sweep_quarter: quarter sweep fo the wings in radians
%   sweep_quarter_vtail: quarter sweep of vertical tail
%   sweep_half_vtail: half sweep of the vertical tail
%   x_a_bar : normalized distance from wing AC to CG (divide by chord)
```

```
%   S_fuselage  : projected side area of fuselage
%   S           : area of wing
%   S_v         : area of vert tail
%   b           : span of wing
%   l_f         : length of fuselage
%   z_w         : vert distance from htail to wing
%   df_max      : max fuselage depth (height)
%   Re_c        : Reynolds number based on chord
%   c_bar       : wing average chord

fuselageCont = C_N_beta_fuselage(S, S_fuselage, b, l_f, Re_c, c_bar);
wingDihedralCont =  C_N_beta_dihedral(C_L, Gamma);
wingSweepCont = C_N_beta_sweep(AR, sweep_quarter, x_a_bar);
[tailCont, k_vtail] = C_N_beta_vtail(S_v, S, b, sweep_quarter_vtail, ...
    sweep_half_vtail, z_w, l_v, df_max, AR, M);

fprintf("Contribution to C_N_beta:\n\tFuselage : %.5f\n\tWing Dihedral : %.5f\n\tWing
Sweep : %.5f\n\tVertical Tail : %.5f\n",...
    fuselageCont, wingDihedralCont, wingSweepCont, tailCont);

C_N_beta= fuselageCont + wingDihedralCont + wingSweepCont + tailCont;
end


function C_N_beta_fuselage = C_N_beta_fuselage(S, S_fuselage, b, l_f, Re_c, c_bar)
%C_N_BETA_FUSELAGE gives C_N_beta of the fuselage in 1/rad
%                   Requires user input for K_b and K_RI from charts
% Built by Matt Liepke for AE 413 Test1
%   S_fuselage  : projected side area of fuselage
%   S           : area of wing
%   b           : span of wing
%   l_f         : length of fuselage
%   Re_c          : Reynolds number based on chord
%   c_bar       : wing average chord

Re_f = Re_c * l_f / c_bar; %for reference
K_RI = str2num(input('K_RI via Fig 3.74?','s'));
K_N = str2num(input('K_N via Fig 3.74?','s'));

C_N_beta_deg = -K_N*K_RI*S_fuselage*l_f / (S*b);
C_N_beta_fuselage = rad2deg(C_N_beta_deg);
end


function C_N_beta_dihedral = C_N_beta_dihedral(C_L, Gamma)
%C_N_BETA_DIHEDRAL provided C_N_beta contribution of the wings via an empirical
formula.
%                  this empirical formula works best for higher aspect ratios
% Built by Matt Liepke for AE 413 Test1
%   C_L     : C_L of the wing
%   Gamma   : wing dihedral in radians
C_N_beta_dihedral =- .075*C_L * Gamma;
end


function C_N_beta_sweep = C_N_beta_sweep(AR, sweep_quarter, x_a_bar)
%C_N_BETA_SWEEP Returns the C_N_beta from sweep via an empirical equation
% Built by Matt Liepke for AE 413 Test1
%   AR      : Aspect ratio of wings
%   sweep_quarter: quarter sweep fo the wings in radians
%   x_a_bar : normalized distance from wing AC to CG (divide by chord)
```

```
C_N_beta_sweep = 1/(4*pi*AR) - tan(sweep_quarter)/(pi*AR*(AR +
4*cos(sweep_quarter)))*...
    (cos(sweep_quarter) - AR/2 - AR^2/(8*cos(sweep_quarter)) -
6*x_a_bar*sin(sweep_quarter)/AR);
end


function [C_N_beta_vtail, k_vtail] = C_N_beta_vtail(S_v, S, b, sweep_quarter_vtail,
sweep_half_vtail, z_w, l_v, df_max, AR, M)
%C_N_BETA_VTAIL Summary of this function goes here
% Built by Matt Liepke for AE 413 Test1
%   S_v : vertical tail surface area
%   S   : surface of the wing
%   b   : span of the wing
%   sweep_quarter_vtail: quarter sweep of vertical tail
%   sweep_half_vtail: half sweep of the vertical tail
%   z_w : vert distance from htail to wing
%   l_v : dist from CG to AC of the horz tail
%   df_max: max fuselage depth (height)
%   AR  : Aspect ratio of wing
%   M   : mach number

V_v = S_v*l_v/(S*b);
sidewash_term = .724 + (3.06*S_v/S)/(1 + cos(sweep_quarter_vtail)) + .4*z_w/df_max +
.009*AR;

%ask for AR_vtail_eff and K
k_vtail = str2num(input('k from Fig 3.75 for vtail? should be around 1: ','s'));
AR_vtail_eff = str2num(input('AR_effective of the tail from 3.77, 7.78, and 3.79?
should be around AR_vtail: ','s'));
C_L_alpha_vtail =C_L_alphawing(AR_vtail_eff, M, 1, sweep_half_vtail);
C_N_beta_vtail = k_vtail*C_L_alpha_vtail*sidewash_term * V_v;

end
```

Static Lateral Stability Derivatives

```
function C_L_beta = C_L_beta(c_r, taper, b, S, AR, C_L_alpha, dihedral, sweep_LE, aoa,
df_max, S_vtail, C_L_alpha_vtail, aoa_vtail, sweep_quarter_vtail, nabla_v, l_v, z_v,
k_vtail)
%C_L_BETA Returns the C_L_beta of all contributions given aircraft
%parameters.
% Built by Matthew Liepke for AE 413 Final Project to work with prior code
% from HW7 (all the functions) and the main code from Test 1 (what calls
% this function)
%   b           : span
%   S           : wing area
%   C_L_alpha   : d(C_L)/d(alpha) for the wing
%   dihedral    : dihedral, in radians
%   sweep       : sweep, in radians
%   AR          : Aspect ratio of wing
%   df_max: max fuselage depth (height)
%   S_vtail     : wing area of vert tail
%   C_L_alpha_vtail: d(C_L)/d(alpha) for the vert tail
%   alpha       : angle of attack , in radians
%   sweep_quarter_vtail: quarter sweep of the vert tail
%   nabla_v     : ratio of dynamic pressure of vtail / wing
%   l_v         : long distance from AC of wing to vert tail
%   v_v         : vert (height) distance from AC of wing to vert tail

res = 10; % number of chord sections per half-wing.  Since this is linear it doesn't
need to be high
```

```
c_fun = c_r*(1+(taper-1)*linspace(0,1,res)); % chord sections
%y_fun = linspace(0,b/2,res); %linear y-dir corresponding to chord sections

C_L_B_dihedralCont = C_L_beta_dihedral(c_fun, b, S, C_L_alpha, dihedral);

C_L_B_sweepCont = C_L_beta_sweep(c_fun, b, S,C_L_alpha, sweep_LE, aoa);

C_L_B_vtail = C_L_beta_vtail(b, AR, S, df_max, S_vtail, C_L_alpha_vtail,...
    aoa, sweep_quarter_vtail, nabla_v, l_v, z_v, k_vtail);

fprintf("C_L_B Contributions: \n\tDihedral: \t%.4f\n\tSweep:
\t%.4f\n\tTail:\t%.4f\n",...
    C_L_B_dihedralCont,C_L_B_sweepCont, C_L_B_vtail);

C_L_beta = C_L_B_dihedralCont + C_L_B_sweepCont + C_L_B_vtail;

end


function C_L_beta_dihedral = C_L_beta_dihedral(chord_fun, b, S, C_L_alpha, dihedral)
%C_L_B_DIHEDRAL Returns the dihedral's contribution to lateral stability
%via strip theory
% Built by Matt Liepke for AE 413 HW6
%   chord_fun   : 1-D array of lengths representing chord progression from
%                   root-> tip
%   b           : span
%   S           : wing area
%   C_L_alpha   : d(C_L)/d(alpha) for the wing
%   dihedral    : dihedral, in radians

y = linspace(0,b/2, length(chord_fun));
C_L_beta_dihedral = -2*dihedral/(S*b) * trapz(y,C_L_alpha*y.*chord_fun);
end


function C_L_beta_sweep = C_L_beta_sweep(chord_fun, b, S, C_L_alpha, sweep, alpha)
%C_L_B_DIHEDRAL Returns the sweep's contribution to lateral stability
%via strip theory
% Built by Matt Liepke for AE 413 HW6
%   chord_fun   : 1-D array of lengths representing chord progression from
%                   root-> tip
%   b           : span
%   S           : wing area
%   C_L_alpha   : d(C_L)/d(alpha) for the wing
%   sweep       : sweep, in radians
%   alpha       : angle of attack , in radians

term1 = -2*alpha*cos(sweep)*sin(sweep)/(S*b);

y_h = sec(sweep) * linspace(0,b/2, length(chord_fun));

term2 = trapz(y_h,C_L_alpha*chord_fun.*y_h);

C_L_beta_sweep = term1*term2;
end


function C_L_beta_vtail = C_L_beta_vtail(b, AR, S, df_max, S_vtail, C_L_alpha_vtail,
alpha, sweep_quarter_vtail, nabla_v, l_v, z_v, k_vtail)
%C_L_B_DIHEDRAL Returns horz tail's contribution to lateral stability via
% Built by Matt Liepke for AE 413 HW6
```

```
%   b           : span
%   AR          : Aspect ratio of wing
%   S           : wing area
%   df_max: max fuselage depth (height)
%   S_vtail     : wing area of vert tail
%   C_L_alpha_vtail: d(C_L)/d(alpha) for the vert tail
%   alpha       : angle of attack , in radians
%   sweep_quarter_vtail: quarter sweep of the vert tail
%   nabla_v     : ratio of dynamic pressure of vtail / wing
%   l_v         : long distance from AC of wing to vert tail
%   z_v         : vert (height) distance from AC of wing to vert tail

sidewash = Sidewash(S_vtail, S, sweep_quarter_vtail, z_v, df_max, AR);

C_L_beta_vtail = -k_vtail * C_L_alpha_vtail * sidewash*nabla_v* (S_vtail/S) *
(z_v*cos(alpha) - l_v*sin(alpha))/b;
end


function Sidewash = Sidewash(S_v, S, sweep_quarter_vtail, z_z, df_max, AR)
%SIDEWASH returns (1 + d(sigma)/d(beta) via empirical formula
% Built by Matt Liepke for AE 413 HW6, taken from work on Test1
%   S_v : vertical tail surface area
%   S   : surface of the wing
%   sweep_quarter_vtail: quarter sweep of vertical tail
%   z_z : vert distance from htail to wing
%   df_max: max fuselage depth (height)
%   AR  : Aspect ratio of wing

Sidewash = .724 + (3.06*S_v/S)/(1 + cos(sweep_quarter_vtail)) + .4*z_z/df_max +
.009*AR;
end
```

Static Control Derivitives

```
function C_L_aileron = C_L_aileron(C_L_alpha_wing, S, b, c_r, lambda, aileron_start,
aileron_end, aileron_start_chord, aileron_end_chord)
%C_L_AILERON Determines the rolling moment derivitave based on aileron
%geometry of the aileron
% C_L_alpha_wing    :   Wing C_L_alpha w/o any aileron deflection
% S                 :   Wing Area
% b                 :   Wing Span
% aileron_start     :   dist from cg -> start of aileron
% aileron_end       :   dist from cg -> end of aileron
% aileron_start_chord : root chord of aileron
% aileron_end_chord :   tip chord of aileron
res = 10; % for numerical integration, does not need to be high due to linear nature
of chord

S_aileron = (aileron_end -
aileron_start)*mean([aileron_start_chord,aileron_end_chord]);
Tau = ControlSurfaceEffectivenessTau(S, S_aileron);

res = 10; % number of chord sections per half-wing.  Since this is linear it doesn't
need to be high

c_fun = c_r*(1+(lambda-1)*linspace(0,1,res)); % chord sections
y_aileron_fun = linspace(aileron_start, aileron_end, res);

int_term = trapz(y_aileron_fun, c_fun.*y_aileron_fun);

C_L_aileron = 2*C_L_alpha_wing*Tau*int_term/(S*b);
```

```
end


function C_N_rudder = C_N_rudder(k_vtail, nabla_vtail, S_rudder, S_vtail,
C_L_alpha_vtail)
%C_N_RUDDER Uses the theoretical formula to find the rudder's contribution
%to C_N.
%   k_vtail     : from chart in 4.1
%   nabla_vtail : dynamic pressure ratio of v_tail to wing
%   S_vtail     : Area of the vertical tail
%   S           : Area of wing
% C_L_alpha_vtail: d(C_L) / d(alpha) for vertical tail

Tau = ControlSurfaceEffectivenessTau(S_vtail, S_rudder);

C_N_rudder = -k_vtail * nabla_vtail * (S_rudder/S_vtail) * C_L_alpha_vtail * Tau;
end


function C_M_elevator = C_M_elevator(nabla_htail, S_elevator, S_htail,
C_L_alpha_htail, l_h, c_bar)
%C_M_ELEVATOR returns the control derivative of C_M with elevator
%deflection
%   nabla_vtail : dynamic pressure ratio of v_tail to wing
%   S_htail     : Area of the horz tail
%   S           : Area of wing
% C_L_alpha_htail: d(C_L) / d(alpha) for horz tail
% l_h           : length from wing AC -> horz. tail AC
% c_bar         : avg. chord of wing
Tau = ControlSurfaceEffectivenessTau(S_htail, S_elevator);

C_M_elevator = -(l_h*S_elevator) / (c_bar * S_htail) * nabla_htail * C_L_alpha_htail *
Tau;
end


function ControlSurfaceEffectivenessTau = ControlSurfaceEffectivenessTau(S_lifting,
S_control)
%CONTROLSURFACEEFFECTIVENESSTAU Gives the Tau parameter from 2.21 via an
%approximate function.
% Created by Matthew Liepke for AE 413 Final project
%   S_lifting   :   Surface of lifting planform (wing, for example)
%   S_control   :   Surface of the control surface planform (aileron, for
%   example)
ratio = S_control/S_lifting;
ControlSurfaceEffectivenessTau = ratio^(.6);
end
```

Published with MATLAB® R2021a